

*Bruno Lourenço
Julia Rebouças
Guilherme Landim
Rodrigo Castelo
Gabriel Felix*

A solid yellow vertical bar is positioned on the left side of the cover, partially behind the title text.

guia Javascript

vol.1

*Desenvolvimento WEB
com Javascript*

ÍNDICE

Cap. 1 Introdução á Tecnologia	02
Cap. 2 Ferramentas para o Desenvolvimento	05
Cap. 3 Passo a Passo de Desenvolvimento	07
Cap. 4 Versionamento de Código	10
Cap. 5 Deploy da Aplicação	01
Cap. 6 Conclusão e Reflexão	01

Capítulo 1: Introdução à Tecnologia

1.1 O que é o JavaScript?

O JavaScript é uma linguagem de programação leve, interpretada e orientada a eventos. Criado em 1995 por Brendan Eich durante seu trabalho na Netscape, o objetivo inicial era possibilitar que os desenvolvedores incorporassem interatividade às páginas da web. Embora tenha surgido como uma solução simples, rapidamente evoluiu para muito mais.

Com o passar do tempo, o JavaScript ultrapassou as limitações dos navegadores. Atualmente, é utilizado para desenvolver desde animações em sites até sistemas back-end robustos, aplicativos móveis e até mesmo jogos. Sua versatilidade e compatibilidade com diversas plataformas o tornaram um dos pilares da tecnologia contemporânea.

1.2 A Relevância do JavaScript no Desenvolvimento Web

A web, como a conhecemos atualmente, seria totalmente diferente sem o JavaScript. Esta linguagem desempenha um papel crucial na interação entre usuários e sistemas online. Confira algumas áreas em que ela é essencial:

- **Interatividade:** O JavaScript possibilita a criação de elementos interativos, como sliders de imagens, menus suspensos e notificações em tempo real.
- **Dinamismo:** Ele permite que as páginas da web respondam ao comportamento do usuário sem precisar recarregar a página completa, graças ao AJAX e às APIs modernas.
- **Versatilidade:** Com frameworks e bibliotecas como React, Angular e Vue.js, o JavaScript simplifica o desenvolvimento de interfaces de usuário ricas e responsivas.

Back-end: Com a chegada do Node.js, o JavaScript também se tornou uma linguagem poderosa para o desenvolvimento de servidores e aplicações escaláveis.

1.3 A Transformação do JavaScript

Desde a sua criação, o JavaScript passou por diversas atualizações e aprimoramentos. Os principais marcos de sua evolução são:

1. 1995 - O JavaScript foi criado pela Netscape.
2. 1997 - A primeira padronização pelo ECMA International estabeleceu a base do ECMAScript (o padrão do JavaScript).
3. 2009 - O lançamento do Node.js expandiu o uso do JavaScript para além dos navegadores, permitindo sua aplicação no back-end.
4. 2015 - A introdução do ECMAScript 6 (ES6) trouxe recursos avançados, como `let`, `const`, classes e arrow functions, modernizando e robustecendo a linguagem.

Atualmente, o JavaScript é constantemente mantido e atualizado, assegurando sua relevância e adaptação às necessidades dos desenvolvedores.

Capítulo 1: Introdução à Tecnologia

1.4 Qual a Importância de Aprender JavaScript?

Aprender JavaScript é uma decisão estratégica para quem quer se destacar no desenvolvimento web ou em outras áreas tecnológicas. Aqui estão algumas razões para considerar o aprendizado dessa linguagem:

- **Demanda elevada:** O JavaScript é amplamente adotado por empresas de todos os portes, desde startups até grandes nomes como Google, Facebook e Amazon.
- **Comunidade vibrante:** A comunidade de desenvolvedores JavaScript é uma das mais extensas e ativas do planeta, garantindo acesso constante a recursos e suporte.
- **Ferramentas e frameworks:** Existe uma ampla variedade de frameworks e bibliotecas que tornam o desenvolvimento com JavaScript mais ágil e eficiente.

Portabilidade: O JavaScript pode ser executado em praticamente qualquer dispositivo conectado à internet, abrangendo desde navegadores até servidores e dispositivos IoT.

1.5 O que Aguardar nos Próximos Capítulos

Agora que entendemos o contexto e a importância do JavaScript, é momento de aplicar a teoria. Nos capítulos seguintes, vamos explorar:

1. As ferramentas essenciais para começar a programar em JavaScript.
2. Um guia completo para criar suas primeiras aplicações.
3. Conceitos mais avançados, como controle de versão e implantação.

Prepare-se para mergulhar em um universo de oportunidades, onde você não apenas aprenderá a programar, mas também a transformar ideias em realidade por meio do JavaScript.

Capítulo 2: Ferramentas para o Desenvolvimento

Antes de iniciar sua programação em JavaScript, é fundamental ter as ferramentas adequadas configuradas. Um ambiente de desenvolvimento bem estruturado não apenas aumenta sua produtividade, mas também previne possíveis problemas. Aqui estão as ferramentas essenciais que você precisa para dar os primeiros passos.

2.1 Navegador Web (Qual a sua importância?)

Um navegador web moderno é fundamental para testar e depurar códigos JavaScript. Navegadores como Google Chrome, Mozilla Firefox e Microsoft Edge possuem consoles de desenvolvedor integrados que facilitam a execução e depuração de JavaScript.

Recomendado: Google Chrome

Como utilizar o console de desenvolvedor:

1. Abra o Google Chrome.
2. Clique com o botão direito em qualquer parte da página e selecione "Inspecionar".
3. Acesse a aba "Console" para executar códigos JavaScript.

Dica: Você pode abrir o console rapidamente usando Ctrl+Shift+I (Windows) ou Cmd+Option+I (Mac).

2.2 Editores de Texto e Ambientes de Desenvolvimento Integrados (IDEs)

Principais opções:

- Visual Studio Code (VS Code): Um dos editores de código mais usados, com suporte a extensões e ótima integração com ferramentas de desenvolvimento.
- Notepad++: Um editor leve e simples para quem prefere algo básico.
- Sublime Text: Um editor minimalista, rápido e personalizável.

Como instalar o VS Code:

1. Acesse o site oficial do [Visual Studio Code](#).
2. Baixe a versão apropriada para o seu sistema operacional (Windows, macOS ou Linux).
3. Execute o instalador e siga as instruções na tela.
4. Após a instalação, abra o VS Code e personalize o ambiente.

2.4 Node.js e npm

Por que usar Node.js e npm?

- Node.js: Permite executar JavaScript fora do navegador, essencial para desenvolvimento backend.
- npm (Node Package Manager): Usado para instalar e gerenciar pacotes e bibliotecas JavaScript.

Como instalar o Node.js (e npm):

1. Acesse o site oficial [Node.js](#).
2. Faça o download da versão recomendada (LTS).
3. Instale seguindo as etapas indicadas.
4. Verifique se a instalação foi concluída corretamente, digitando no terminal:

```
bash
node -v
npm -v
```



Isso exibirá as versões do Node.js e do npm instaladas no seu computador.

Capítulo 2: Ferramentas para o Desenvolvimento

2.5 Extensões Indispensáveis para o VS Code

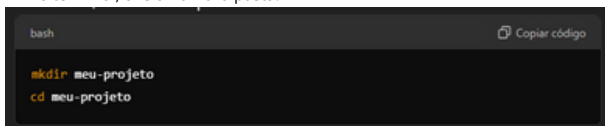
O VS Code possui um marketplace de extensões que podem melhorar significativamente sua experiência ao programar. Aqui estão as mais úteis:

1. ESLint:
 - Analisa seu código e destaca erros diretamente no editor.
 - Para instalar, vá até a aba de extensões no VS Code, pesquise por "ESLint" e clique em Instalar.
2. Prettier:
 - Um formatador automático de código que mantém seu código limpo e padronizado.
 - Pesquise por "Prettier" nas extensões e instale.
3. Live Server:
 - Cria um servidor local para você visualizar mudanças no navegador em tempo real.
 - Após instalar, basta clicar com o botão direito no arquivo HTML e escolher "Open with Live Server".

2.6 Extensões Indispensáveis para o VS Code

Passos finais:

1. Criação de um projeto básico:
 - No terminal, crie uma nova pasta:



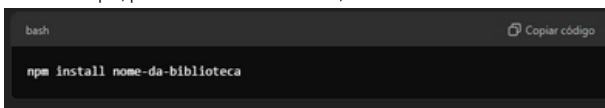
```
bash
mkdir meu-projeto
cd meu-projeto
```

- Inicialize o projeto com npm:



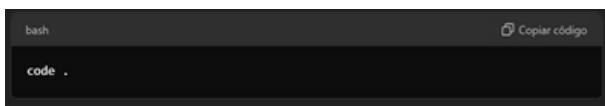
```
bash
npm init -y
```

1. Instale bibliotecas e pacotes conforme necessário:
 - Por exemplo, para instalar uma biblioteca, use:



```
bash
npm install nome-da-biblioteca
```

1. Abra o projeto no VS Code:
 - No terminal, digite:



```
bash
code .
```

Com essas ferramentas configuradas, você estará pronto para explorar os fundamentos do JavaScript e começar a criar projetos incríveis. Nos próximos capítulos, aprenderemos como utilizá-las na prática!

Capítulo 3: Passo a Passo de Desenvolvimento

3.1 Inicializando o projeto

Passos iniciais:

1. Criação de um projeto básico:
 - No terminal, crie uma nova pasta:

```
bash Copiar código
mkdir meu-projeto
cd meu-projeto
```

- Inicialize o projeto com npm:

```
bash Copiar código
npm init -y
```

2. Instale o eslint

```
bash Copiar código
npm install eslint --save-dev
```

3. Inicialize o ESLint no projeto:

```
bash Copiar código
npx eslint --init
```

1. Após configurado, use o ESLint para verificar erros no código:

```
bash Copiar código
npx eslint script.js
```

Capítulo 3: Passo a Passo de Desenvolvimento

3.2 Estruturando o HTML da calculadora

Aqui está o arquivo index.html, que contém a interface básica da calculadora.

```
index.html X
index.html > ...
1 <!DOCTYPE html>
2 <html lang="pt-BR">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Calculadora Simples</title>
7 </head>
8 <body>
9   <h1>Calculadora</h1>
10  <p>Digite dois números e escolha uma operação:</p>
11
12  <!-- Campos para entrada de números -->
13  <input type="number" id="num1" placeholder="1º número">
14  <input type="number" id="num2" placeholder="2º número">
15
16  <!-- Botões para operações -->
17  <div>
18    <button id="add">Somar</button>
19    <button id="subtract">Subtrair</button>
20  </div>
21
22  <!-- Área para exibição do resultado -->
23  <p>Resultado: <span id="output">0</span></p>
24
25  <script src="script.js"></script>
26 </body>
27 </html>
28
```

Deve estar parecendo com isso:

Calculadora

Digite dois números e escolha uma operação:

Resultado: 0

Capítulo 3: Passo a Passo de Desenvolvimento

3.3 Adicionando a Lógica com JavaScript

Aqui está o arquivo script.js, que contém a lógica básica da calculadora.

```
index.html  JS script.js  X
# script.js > ...
1 // Captura os elementos do DOM
2 const num1 = document.getElementById("num1");
3 const num2 = document.getElementById("num2");
4 const addButton = document.getElementById("add");
5 const subtractButton = document.getElementById("subtract");
6 const output = document.getElementById("output");
7
8 // Função para somar os números
9 addButton.addEventListener("click", () => {
10   const n1 = parseFloat(num1.value);
11   const n2 = parseFloat(num2.value);
12
13   // Verifica se os números são válidos
14   if (isNaN(n1) || isNaN(n2)) {
15     output.textContent = "Insira números válidos!";
16   } else {
17     output.textContent = n1 + n2;
18   }
19 });
20
21 // Função para subtrair os números
22 subtractButton.addEventListener("click", () => {
23   const n1 = parseFloat(num1.value);
24   const n2 = parseFloat(num2.value);
25
26   // Verifica se os números são válidos
27   if (isNaN(n1) || isNaN(n2)) {
28     output.textContent = "Insira números válidos!";
29   } else {
30     output.textContent = n1 - n2;
31   }
32 });
33
```

Sua interface agora contém lógica javascript e em instantes deverá estar efetuando somas e subtrações. Antes disso no HTML você precisa importar o arquivo script.js para poder funcionar.

Calculadora

Digite dois números e escolha uma operação:

1

2

Somar

Subtrair

Resultado: 3

Agora o arquivo JavaScript (script.js) será carregado e executado corretamente no navegador, garantindo que as funcionalidades da calculadora funcionem como esperado. 😊

Capítulo 4: Versionamento de Código

4.1 Introdução ao Versionamento de Código:

O versionamento de código é uma prática essencial no desenvolvimento de software moderno. Com ferramentas como o Git, é possível:

- Rastrear alterações no código;
- Facilitar o trabalho em equipe;
- Reverter problemas causados por bugs ou conflitos.

4.2 Instalando e Configurando o Git:

Instalação:

1. Baixe o Git:

- Download para Windows
- Download para macOS
- Download para Linux

2. Verifique a instalação:

```
git --version
```

2.1 Verifique a instalação:

Configure o nome e o e-mail:

```
git config --global user.name "Seu Nome"
git config --global user.email "seuemail@exemplo.com"
```

4.3 Principais Comandos do Git:

Criando um Repositório:

1. Inicialize um repositório Git:

```
git init
```

2. Adicione um arquivo:

```
echo "console.log('Hello, World!');" > app.js
git add app.js
git commit -m "Primeiro commit: adiciona app.js"
```

Capítulo 4: Versionamento de Código

4.3 Trabalhando com Branches:

1.Crie uma nova branch:

```
git branch nova-feature  
git checkout nova-feature
```

ou outra opção:

```
git checkout -b nova-feature
```

2.Mescle as alterações da branch principal:

```
git merge main
```

4.4 Conflitos de Merge:

Quando dois desenvolvedores alteram a mesma linha em um arquivo, ocorre um conflito.

Resolva editando manualmente o arquivo afetado e, em seguida, finalize com:

```
git add arquivo.js  
git commit -m "Resolve conflito no arquivo.js"
```

4.5 Integração com Plataformas Remotas:

Clonando um Repositório:

Clone um repositório remoto:

```
git clone https://github.com/usuario/repo.git
```

1. Enviando alterações:

Conecte o repositório remoto:

```
git remote add origin https://github.com/usuario/repo.git
```

Envie alterações:

```
git push -u origin main
```

Capítulo 4: Versionamento de Código

4.6 Fluxos de Trabalho em Projetos JavaScript:

Configurando um Ambiente de Desenvolvimento:

1. Crie um projeto:

```
mkdir projeto-js
cd projeto-js
npm init -y
```

2. Adicione arquivos ao controle de versão:

```
echo "node_modules" > .gitignore
git add .
git commit -m "Inicializa projeto com npm"
```

Exemplo na prática:

Implementando uma Função em JavaScript:

Crie o arquivo `script.js` com o seguinte código:

```
script.js
C:\Users\gabriel> Downloads > script.js > subtractButton.addEventListener("click") callback
1 // Captura os elementos do DOM
2 const num1 = document.getElementById("num1");
3 const num2 = document.getElementById("num2");
4 const addButton = document.getElementById("add");
5 const subtractButton = document.getElementById("subtract");
6 const output = document.getElementById("output");
7
8 // Função para somar os números
9 addButton.addEventListener("click", () => {
10   const n1 = parseFloat(num1.value);
11   const n2 = parseFloat(num2.value);
12
13   // Verifica se os números são válidos
14   if (isNaN(n1) || isNaN(n2)) {
15     output.textContent = "Insira números válidos!";
16   } else {
17     output.textContent = n1 + n2;
18   }
19 });
20
21 // Função para subtrair os números
22 subtractButton.addEventListener("click", () => {
23   const n1 = parseFloat(num1.value);
24   const n2 = parseFloat(num2.value);
25
26   // Verifica se os números são válidos
27   if (isNaN(n1) || isNaN(n2)) {
28     output.textContent = "Insira números válidos!";
29   } else {
30     output.textContent = n1 - n2;
31   }
32 });
33
```

Adicione e registre estas **alterações**:

```
git add script.js
git commit -m "Chore: Adicionando arquivo script.js"
```

Capítulo 6: Conclusão e Reflexão

Em um cenário digital cada vez mais dinâmico e interativo, o JavaScript se mantém como uma das linguagens mais poderosas e relevantes para o desenvolvimento de software. Desde suas origens simples até sua adoção em sistemas complexos, o JavaScript tem mostrado sua versatilidade, sendo utilizado para criar desde pequenas interações até grandes plataformas e aplicações. Seu papel na transformação da web, tanto no front-end quanto no back-end, é inegável, e sua constante evolução assegura que continuará sendo uma ferramenta essencial no desenvolvimento de novas soluções tecnológicas. Para os desenvolvedores, aprender JavaScript é mais do que uma habilidade técnica — é uma chave para se inserir e se destacar em um dos setores mais inovadores e em constante crescimento da tecnologia moderna.

